arXiv:2404.01744v5 [cs.CL] 16 Apr 2024

Octopus v2: On-device language model for super agent

Wei Chen† *
Stanford University
{weichen6}@stanford.edu

Zhiyuan Li†
Stanford University
{zhiyuan8}@stanford.edu

Abstract

Language models have shown effectiveness in a variety of software applications, particularly in tasks related to automatic workflow. These models possess the crucial ability to call functions, which is essential in creating AI agents. Despite the high performance of large-scale language models in cloud environments, they are often associated with concerns over privacy and cost. Current on-device models for function calling face issues with latency and accuracy. Our research presents a new method that empowers an on-device model with 2 billion parameters to surpass the performance of GPT-4 in both accuracy and latency, and decrease the context length by 95%. When compared to Llama-7B with a RAG-based function calling mechanism, our method enhances latency by 35-fold. This method reduces the latency to levels deemed suitable for deployment across a variety of edge devices in production environments, aligning with the performance requisites for real-world applications.

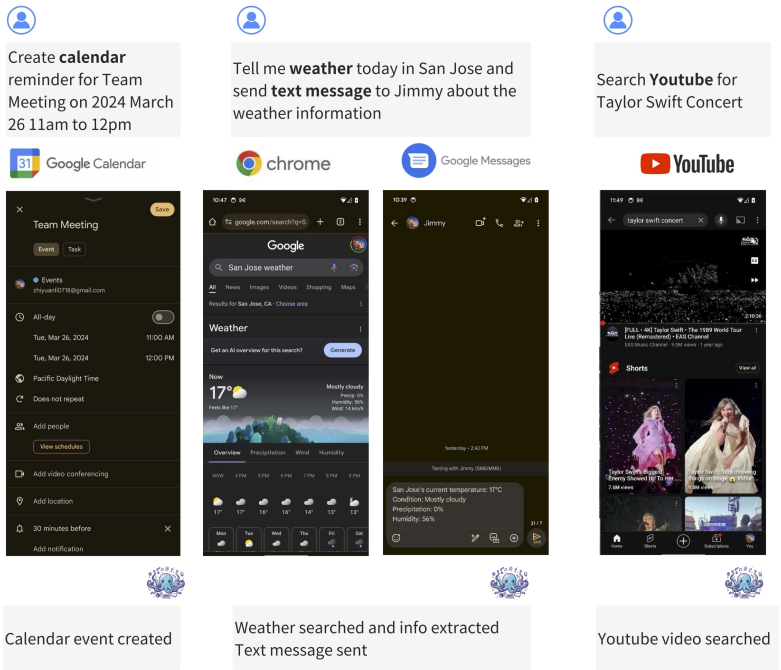

Figure 1: Automated workflow in smartphone using Octopus model.

*Corresponding author, † equal contribution

Preprint. Under review.

1 Introduction

Large language models have demonstrated impressive capabilities in function calling, significantly contributing to AI agents' growing presence in the software industry [50, 3, 11, 7, 8]. The advancement in AI agents is rapid, highlighted by the AI assistant tools like MultiOn [9] and Adept AI [25], and AI consumer products like Rabbit R1 [26] and Humane AI Pin [5], which are gaining traction in the consumer sector. Research into AI agents has been robust, witnessing developments in the chain of thought [58, 16] and enhanced prompting techniques [51]. Moreover, the rise of multi-agent systems [53, 43, 39, 29] marks a novel trend in the industry, showcasing the use of language models to develop dependable software that empowers users [54, 38]. These innovations leverage the API calling [56, 12, 49] and reasoning abilities [40, 36] of large, cloud-based language models to convert human natural language instructions into actionable commands. Despite the considerable progress in creating valuable AI agents, reliance on cloud models raises issues concerning privacy, inference costs, and the need for Wi-Fi connectivity [57, 22].

The cost of using large language models like Google's Gemini family models [45] and OPENAI's GPT series models [33, 34, 2, 1] can be substantial; for example, an hour-long interaction with an AI bot might cost around 0.24 USD according to the GPT-4 API price. When it comes to function calling, employing RAG-based [17, 27, 19, 15] or context-augmented [35] methods requires processing about 1000 tokens for each call, resulting in costs of approximately 0.01 USD. In practical applications, where hundreds of function calls may be made, the cumulative cost can be much. Additionally, the potential for privacy violations deters many from using GPT-4, amid concerns that sensitive information might be exposed.

To mitigate costs and enhance privacy, there is a trend towards creating smaller models for deployment on edge devices like smartphones, cars, VR headsets, and personal computers [52, 6, 21, 18, 55, 13, 41]. However, edge computing-based models face challenges with slower latency, far from production readiness, and the limited battery life of edge devices further complicates continuous interaction. Research shows that energy consumption reaches 0.1J per token for 1 billion parameter models [23]. Therefore, employing a 7B parameter model for function calls with traditional retrieval-augmented methods would consume 700J per call, roughly 1.4% of a 50kJ iPhone battery, limiting to around 71 function calls.

Smaller models often fall short in reasoning tasks and demand extensive tuning for effective function calling. To address these issues, we developed a method to enhance both accuracy and latency for function calling on 2B parameter models on devices, achieving state-of-the-art (SOTA) results. This approach involves tokenizing the core function's name and fine-tuning the model with *functional tokens*. Fine-tuning with these tokens allows the model to understand software application capabilities with additional special tokens, learning to map function descriptions to specific tokens. In the inference phase, the model uses *functional tokens* to achieve better performance in function calling compared to GPT-4. We present a 2B parameter model fine-tuned from Gemma 2B [10], saving over 95% context length during model inference. For iPhone use, this enables 37 times more function calls with the same battery and reduces latency by approximately 35 times per function call.

2 Related works

Deployment of on-device language models Due to memory limitations and lower inference speeds, deploying larger models on edge devices like PCs or smartphones is challenging. Nonetheless, efforts to deploy smaller-scale Large Language Models (LLMs) to edge devices are underway. Open-source models of manageable sizes, such as Gemma-2B, Gemma-7B, StableCode-3B [31], and Llama-7B [47], have been introduced. To enhance these models' inference speed on devices, research initiatives like Llama cpp [24] have been developed. The MLC LLM framework [46] allows the operation of 7B language models on mobile phones and other edge devices, demonstrating compatibility across various hardware, including AMD, NVIDIA, Apple, and Intel GPUs.

Function calling in language models Rapid advancements have been observed in the function-calling capabilities of smaller-scale models. Projects such as NexusRaven [42], Toolformer [37], ToolAlpaca [44], Gorrilla [30], ToolLlama [32] and Taskmatrix [20] have demonstrated that 7B and 13B models can call external APIs with efficacy comparable to GPT-4. The pioneering Octopus v1 project even enabled a 2B model to perform on par with GPT-4. This body of work utilizes a

RAG-based method for function calling, where the model retrieves relevant functions from a large pool based on the user's query, then generates a response using these functions as context.

Fine-tuning and adaptors of language models Fine-tuning language models has become a prevalent practice, with various efforts dedicated to this endeavor. LoRA [14] is often the method of choice for training models under GPU resource constraints. We use both full model training and LoRA training in our work, and compare their performance. A notable benefit of LoRA is its facilitation of extended functionalities in models, suggesting its potential to adapt our current framework for a broad range of applications.

3 Methodology

In this section, we detail the primary methodology implemented in our models, followed by the dataset collection process essential for fine-tuning these models. We illustrate this through examples drawn from the Android API. Subsequently, we delve into the specifics of our model training approach.

3.1 Causal language model as a classification model

To successfully invoke a function, it's essential to accurately select the appropriate function from all available options and to generate the correct function parameters. This entails a two-stage process: a function selection stage and a parameter generation stage. The initial step involves understanding the function's description and its arguments, using information from the user's query to create parameters for an executable function. A direct strategy might combine a classification model with a causal language model. We can envision the N available functions as a selection pool, transforming the selection challenge into a softmax classification problem.

One straightforward method for classification could be retrieval-based document selection, identifying the function that most closely matches the user's query by semantic similarity. Or we can use a classification model to map the query to a specific function name. Alternatively, autoregressive models, such as a GPT model, can predict the correct function name from the user's query within the context of potential functions. Both approaches essentially divide the task into two parts, potentially requiring two models, π_1 and π_2:

$$P(f|q) = P(f|q;\pi_1), \quad P(\text{params}\,|f,q) = P(\text{params}\,|f,q;\pi_2), \tag{1}$$

where q denotes the query, f signifies the selected function name, and *params* represent the parameters for the chosen function. Driven by the principles of multitask learning/meta learning [4], and to achieve quicker inference speeds and system convenience, we pursued a unified GPT model strategy, setting $\pi_1 = \pi_2 = \pi$. Therefore, we redefine our objective as:

$$P(f, \text{params}\,|q) = P(f|q;\pi)P(\text{params}\,|f,q;\pi). \tag{2}$$

For $P(f|q;\pi)$, the traditional method involves retrieving relevant functions and providing context about several pertinent functions to deduce the optimal function names. In most use cases, the set of possible function names is fixed. When utilizing a language model to formulate a function name, multiple tokens must be generated to form one function name, which can lead to inaccuracies. To mitigate such errors, we propose designating functions as unique *functional tokens*. For example, in a pool of N available functions, we assign token names ranging from `<nexa_0>` to `<nexa_N-1>` to symbolize these functions. This transforms the prediction task for function names into a single-token classification among the N functional tokens, enhancing the accuracy of function name prediction while simultaneously reducing the number of tokens required. To implement this, we introduce new special tokens from `<nexa_0>` to `<nexa_N-1>` into the tokenizer and modify the architecture of the pretrained model to expand the language head by an additional N units. Thus, for function name prediction, we utilize the language model to pinpoint the correct function among the N functional tokens through argmax probability selection.

To choose a correct *functional token*, the language model must grasp the meaning associated with that token. We decided to incorporate the function descriptions into the training dataset, enabling the model to learn the importance of these specialized tokens. We designed a prompt template that accommodates three different response styles, facilitating parallel and nested function calls. Detailed examples of the dataset are provided in the Appendix.

Below is the query from the users, please choose the correct function and generate the parameters to call the function.
Query: {query}

for single function call
Response: <nexa_i>(param1, param2, ...)<nexa_end>

for parallel function call
Response:<nexa_i>(param1, param2, ...);<nexa_j>(param1, param2, ...)<nexa_end>

for nested function call
Response:<nexa_i>(param1, <nexa_j>(param1, param2, ...), ...)<nexa_end>
Function description: {function_description}

This methodology presents an additional critical benefit. After the model is fine-tuned to understand the significance of functional tokens, it can conduct inference by employing the added special token, <nexa_end>, as the **early stopping criterion**. This strategy negates the necessity to analyze tokens from function descriptions, removing the retrieval of relevant functions and the processing of their descriptions. Consequently, this considerably diminishes the number of tokens needed to accurately identify a function name. The difference between the conventional retrieval-based method and our current proposed model is shown in Figure (2).

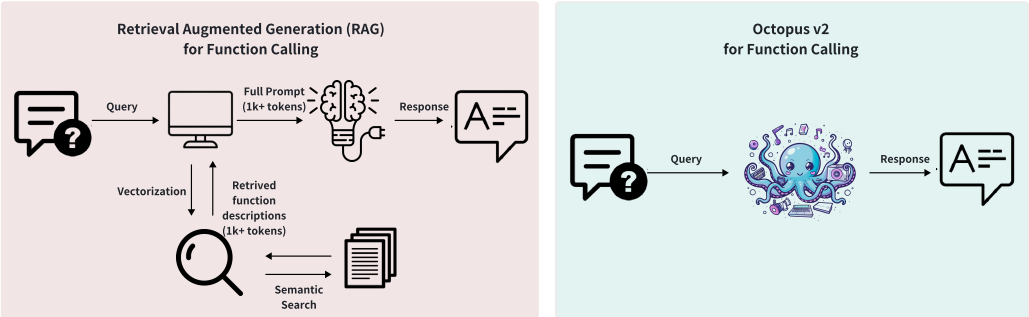

Figure 2: The comparison of the retrieval-based function calling process and the function calling process of the Octopus model.

3.2 Functional token

The critical innovation presented in this paper is the introduction of the functional token. Drawing parallels to the tokenization of natural language, we propose representing specific functions as functional tokens. We introduce a training methodology for these tokens, inspired by techniques used in natural language models for handling rare words. This training approach is inspired by the word2vec [28] framework, where contextual words enrich a token's semantic representation. For instance, pretrained language models may initially struggle to recognize specialized terms such as *PEGylation* and *Endosomal Escape* from the domain of chemistry. However, these models can learn such terms through causal language modeling, leveraging corpora that include these specialized terms. Similarly, functional tokens can be learned using comparable methods. Our experiments have not identified any limitations on the number of functional tokens that can be defined, allowing users to map any specific function to a token.

By incorporating the functional token, we aim to assign the same importance to functional tokens as other linguistic tokens. Unlike conventional linguistic tokens, our functional token does not possess inherent natural language meaning; instead, it represents specific actions encapsulated within the model. For smaller-scale models like Google Gemma 2B, conducting function calling can be challenging, although they excel at typical language completion tasks. The functional token enables

the language model to transform a functional calling method into a standard completion task. As we define the actions for the language model, we can also characterize our model as a *small action model*. The integration of the functional token allows the model to concentrate on a fixed set of actions and perform effectively on these tasks.

3.3 Dataset collection

This segment outlines our methodology for assembling datasets of superior quality for the phases of training, validation, and testing. It also describes the organized process we utilized to arrange the dataset for efficient training purposes.

API Collection As an example, we start with Android APIs. Our selection criteria encompass usability, usage frequency, and the complexity of technical implementation. We ultimately gather 20 Android APIs and organize them into three separate categories, ensuring that each function can be realistically executed on devices through Android app development, provided the developer possesses the necessary system permissions. Additionally, we also compile APIs available in vehicles. More examples can be found in the Appendix.

1. **Android system API** This category includes APIs for system-level functions essential for basic mobile operations, such as making calls, texting, setting alarms, modifying screen brightness, creating calendar entries, managing Bluetooth, enabling do-not-disturb mode, and taking photos. We exclude highly sensitive tasks like accessing system state information or changing accessibility settings.
2. **Android App API** Our research examines APIs from pre-installed Google apps on Android devices, such as YouTube, Google Chrome, Gmail, and Google Maps. We explore functionalities like accessing trending news, retrieving weather updates, searching for YouTube content, and map navigation.
3. **Android smart device management API** Our focus extends to the Google Home ecosystem, which comprises a wide range of smart home devices with significant market presence. Our aim is to improve smart device management via APIs, covering functions like adjusting a Nest Thermostat, managing media playback on a Google Nest device, and controlling door locks using the Google Home App.

Dataset generation Our approach is depicted in Figure (3), showcasing the steps involved in assembling the dataset. The creation of the dataset involves three key phases: (1) generating relevant queries and their associated function call arguments; (2) developing irrelevant queries accompanied by suitable function bodies; and (3) implementing binary verification support through Google Gemini.

1. **Google Gemini Generated Query and Function Call** Creating a high-quality dataset hinges on formulating well-defined queries and accurate function call arguments. Our strategy emphasizes generating positive queries that a single API can resolve. With a query and predetermined API descriptions in hand, we utilize a subsequent Google Gemini API call to produce the required function call arguments.
2. **Negative Samples** To enhance the model's analytical skills and practical application, we incorporate examples from both positive and negative datasets. The equilibrium between these sets is represented by the ratio $\frac{M}{N}$ in Figure 3, fundamental to our experimental methodology. Specifically, we select M and N to be equal, each assigned a value of 1000.

Dataset Verification Despite the advanced capabilities of large language models such as OpenAI's GPT-4 and Google's Gemini, there remains a noticeable rate of errors, particularly in the generation of function call arguments. These errors may manifest as missing arguments, incorrect argument types, or misinterpretations of the intended query. To mitigate these shortcomings, we have introduced a verification mechanism. This system allows Google Gemini to evaluate the completeness and accuracy of its generated function calls, and should the output be found lacking, it initiates a regeneration process.

3.4 Model development and training

We employ the Google Gemma-2B model as the pretrained model in our framework. Our approach incorporates two distinct training methodologies: full model training and LoRA model training. For

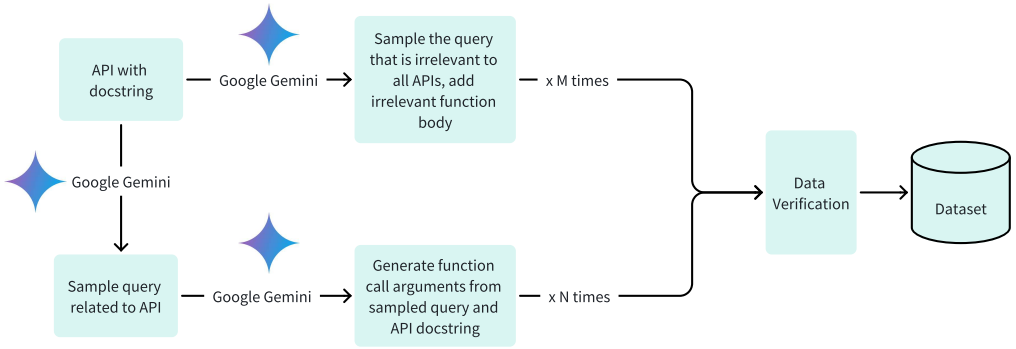

Figure 3: The Process of Generating the Dataset: This involves two critical stages: (1) creation of solvable queries specific to certain APIs and the generation of appropriate function calls for them, and (2) creation of unsolvable queries, complemented by unrelated function bodies. Incorporating a binary validation mechanism for rigorous validation ensures the collection of an optimized training dataset, poised to significantly improve model functionality.

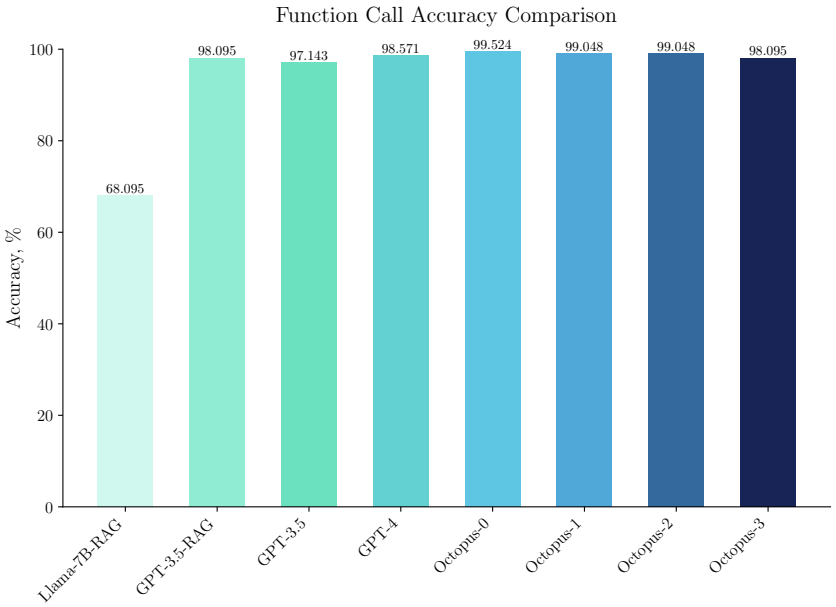

Figure 4: Accuracy Plot for Benchmark: This analysis includes the Llama-7B with RAG, GPT-3.5 with RAG, GPT-3.5, GPT-4, and the Octopus series models, labeled Octopus-0, Octopus-1, Octopus-2, and Octopus-3. The distinction among Octopus models arises from the dataset size and training methodology. The original Octopus-0 model was trained using the full model approach with 1K data points per API. Octopus-1, while also utilizing 1K data points per API, was trained using the LoRA method. Octopus-2 and Octopus-3 followed the full model training but with reduced data points of 500 and 100, respectively. For comprehensive differences among these models, refer to Table (1).

full model training, we utilize an AdamW optimizer with a learning rate set at `5e-5`, a warm-up step of 10, and a linear learning rate scheduler. The same optimizer and learning rate configuration are applied to LoRA training. We specify the LoRA rank as 16 and apply LoRA to the following modules: `q_proj`, `k_proj`, `v_proj`, `o_proj`, `up_proj`, `down_proj`. The LoRA alpha parameter is set to 32. For both training methods—full model and LoRA—we set the number of epochs to 3.

4 Experiments

Our study conducts a comprehensive evaluation of language model capabilities via an extensive benchmarking approach, aimed at assessing their effectiveness in generating accurate function calls. Initially, we compare our model's accuracy and response time against premier models in the field, namely GPT-4 (checkpoint: *gpt-4-0125-preview*) and GPT-3.5 (checkpoint: *gpt-3.5-turbo-0125*).

In the next phase, we explore the efficacy of the RAG technique, renowned for its ability to reduce incorrect outputs (hallucinations) and latency by equipping language models with a concise selection of potential functions. Through the integration of Meta's `FAISS` for semantic search, we enhance the function call description retrieval process, opting for the top 5 descriptions to navigate context length constraints seen in models like Meta's Llama-7B and OpenAI's GPT-3.5.

Subsequently, we analyze the impact of training dataset size and model training methods on performance metrics.

4.1 Android function calls

To illustrate our model's application, we select Android system function calls as a case study, focusing on accuracy and latency in function call generation. Initially, we chose 20 Android APIs as our dataset foundation. We adopted two distinct methods for generating function call commands. For details on API design, see Appendix. The first method involves a RAG approach to identify top similar function descriptions based on user queries, which the language model then uses, along with the user query, to generate the expected function call commands. We detail the various models employed during this evaluation.

Utilizing Google Gemini, we sample relevant queries for the selected Android function calls and manually label the ground truth as the evaluation dataset. We document our benchmark results, focusing on two critical metrics: accuracy and latency, as illustrated in Figure (4) and Figure (5) respectively.

Llama-7B RAG Evaluation Initially, the pretrained Llama-7B model showed limited ability in generating the expected outcomes, leading us to employ a Llama-7B variant fine-tuned for function call generation [48]. For the Llama-7B assessment, we applied the RAG method without strict output format requirements, considering responses with missing parentheses as correct. This evaluation was conducted on a single NVIDIA A100 machine, with all results from Llama-7B compared against the ground truth. The primary errors were incorrect function name selection and erroneous parameter generation. Despite employing few-shot learning to guide the model towards accurate function generation, the performance was modest, with an accuracy rate of 68.095% when overlooking format requirements and a latency of 13.46 seconds, excluding model loading time. To improve latency, we implemented optimizations such as flash attention and a fast tokenizer.

GPT-3.5 RAG Evaluation Similar to the approach with Llama-7B, we utilized GPT-3.5 for response generation, employing the same semantic search strategy for context acquisition. To enhance GPT-3.5's performance, we designed a specific prompt style, incorporating one-shot learning to improve accuracy further.

In this benchmark test, an impressive accuracy of 98.095% was achieved, leveraging the *gpt-3.5-turbo-0125* checkpoint known for its optimization in function calling tasks. The latency was significantly improved to 1.97 seconds for generating a single function call, a notable enhancement over the Llama-7B model's performance. This improvement in speed is primarily attributed to the efficiency of the language model inference, as the RAG component remained consistent. GPT-3.5's quicker response may be due to OpenAI's use of multiple GPUs or a more advanced inference infrastructure. Further analysis revealed that a significant portion of the time was spent on content retrieval, despite only needing to fetch 5 function descriptions from a pool of 20. To optimize latency, all function descriptions' embeddings were precomputed using OpenAI's *text-embedding-3-large* endpoint, with `IndexFlatL2` employed for search indexing and parallel computation on multicore CPUs to enhance speed.

Below, you are presented with {n_candidates} candidate functions. Your task is to analyze a specific query to determine which of these functions most appropriately addresses the query. Then, construct the correct function call with all necessary parameters, adhering to proper syntax.
Format for function call:
function_name(param1, param2, ...)
Candidate Functions: {candidates}

def irrelevant_function():
    '''If user query is not related to any of the predefined functions, this function will be called.
Args: Returns:'''

Query:{query}
Example Scenario:
Query: "Change user's display mode to dark theme."
Given Functions: switch_theme(theme), set_brightness(level), irrelevant_function()
Output: switch_theme("dark")

Your goal is to select the most suitable function out of the n_candidates candidates and generate an accurate function call that directly addresses the query. Ensure the output is a syntactically valid function call. Only return the function call.

GPT-3.5 and GPT-4 Evaluations In efforts to further reduce latency for GPT-3.5 and GPT-4, we included all 20 function descriptions directly in the context, bypassing the RAG method to avoid microservices interactions and their associated IO-bound overheads. This adjustment yielded a latency reduction to 1.18 seconds for GPT-3.5. The prompt template mirrored that of the GPT-3.5 RAG, with the addition of more candidate functions. However, accuracy slightly declined to 97.143%, possibly due to diminished language model effectiveness with longer text inputs. Conversely, GPT-4 exhibited superior accuracy at 98.571% and even lower latency than GPT-3.5, despite expectations of GPT-4 being a larger model. This performance, evaluated on March 18 at 2 PM PDT, might reflect variances in API traffic or hardware configurations between the two models. GPT-4's enhanced performance suggests OpenAI could be allocating more GPU resources to it or that it experiences less demand compared to GPT-3.5.

Octopus model Now, we present the octopus model with 1000 data points sampled for each API. And we observe a 99.524% accuracy in our evaluation dataset. Moreover, the prompt used for this method is as simple as:

"Below is the query from the users, please call the correct function and generate the parameters to call the function. Query: {user_query} Response:"

In our approach, incorporating function information directly into the context is unnecessary, as the Octopus model has already learned to mapping functional tokens to corresponding function descriptions, thereby conserving a significant number of tokens for processing. Given its compact size and the brevity of the context required, the Octopus model demonstrates a reduced latency of 0.38 seconds. To maintain an equitable comparison, we adhered to the same benchmark settings used for the Llama7B evaluation, such as incorporating flash attention and not using quantization. Furthermore, we explored the deployment of our Octopus 2B model on mobile devices through quantization. By precomputing the state for the fixed prefix—`"Below is the query from the users, please call the correct function and generate the parameters to call the function. Query:"`—our on-device model achieves remarkable performance, completing a function call within 1.1 to 1.7 seconds for typical queries of 20 to 30 tokens using a standard Android phone.

4.2 Extension to Vehicle, Yelp, and DoorDash function sets

In addition to Android function calls, we expanded our evaluation to include 20 vehicle function calls, showcasing the algorithm's adaptability to diverse use cases. For vehicle functions, we focused

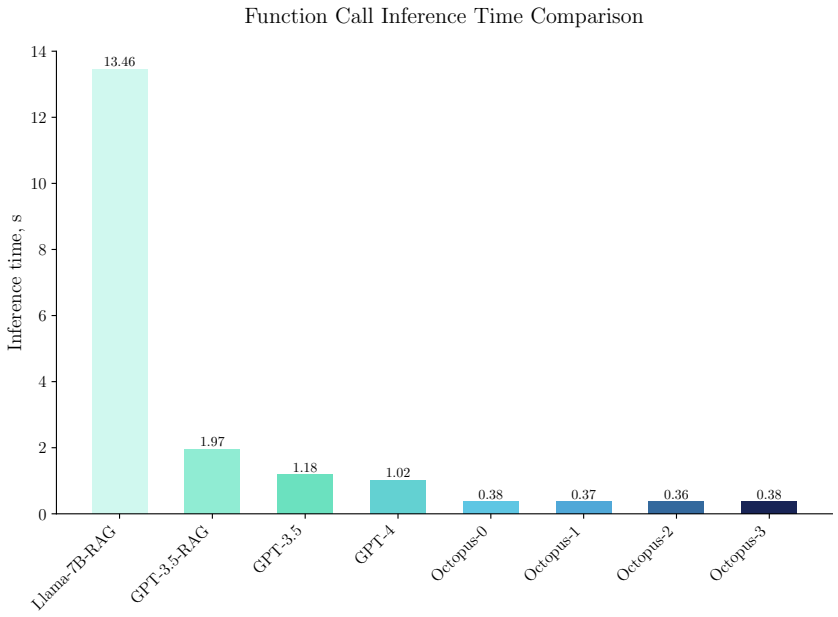

Figure 5: Latency Plot for Benchmark Models: This analysis includes the Llama-7B with RAG, GPT-3.5 with RAG, GPT-3.5, GPT-4, and the Octopus series models, labeled Octopus-0, Octopus-1, Octopus-2, and Octopus-3. The distinction among Octopus models arises from the dataset size and training methodology. The original Octopus-0 model was trained using the full model approach with 1K data points per API. Octopus-1, while also utilizing 1K data points per API, was trained using the LoRA method. Octopus-2 and Octopus-3 followed the full model training but with reduced data points of 500 and 100, respectively. For comprehensive differences among these models, refer to Table (1).

on essential control methods such as volume adjustment, air conditioning, and seat positioning. We conducted benchmarks for vehicle functions paralleling the Android function evaluation, observing consistent performance patterns. Details on vehicle functions are provided in the Appendix, enabling users to customize a new set of functional APIs for their specific needs. Furthermore, tests conducted with Yelp and DoorDash APIs confirmed a similar performance, underscoring our method's versatility across various function sets.

4.3 Full and partial training datasets

The Octopus model demonstrates exceptional performance with 1,000 data points sampled for each API during its training phase. However, for training a new set of functions, cost efficiency becomes a consideration, given the need to generate a training dataset. In our analysis, generating 1,000 data points for a single API incurs a cost of 0.0224 USD, representing the investment required to train an Octopus-0 model for one specific function. By evaluating the Octopus-0, Octopus-2, and Octopus-3 models, we discern that sampling only 100 data points for one API can still achieve an accuracy of 98.095%, as depicted in Figure (4). Therefore, for individuals seeking to train their own Octopus model using our framework, we recommend a dataset size ranging from 100 to 1,000 data points.

4.4 Full training and LoRA training

LoRA plays a crucial role in our framework, particularly when integrating the Octopus model across multiple applications to ensure smooth computation. Instead of employing full models for each API set, we opt for diverse LoRA trainings tailored to the specific function setups of different apps. As Figure (4) illustrates, switching to Lora training results in a minor accuracy decrease. Nonetheless, the maintained high accuracy levels are sufficiently robust for production deployment.

Model name	Training Dataset Size	Training configuration
Octopus-0	1K per API	Full model training
Octopus-1	1K per API	LoRA, rank = 16, lora_alpha = 32, applied on "q_proj", "k_proj", "v_proj", "o_proj", "up_proj", "down_proj"
Octopus-2	500 per API	Full model training
Octopus-3	100 per API	Full model training

Table 1: Configuration of the four different octopus models.

4.5 Parallel and nested function call

For the benchmark test above, we indicate that they are intended for the single function call. To enable the parallel function call and the nested function call, we need to prepare 4K data points for each API so that the accuracy can reach the same level as the single function call.

4.6 Weighted loss function for special tokens

A distinctive aspect of our approach involves incorporating numerous special tokens into the tokenizer and expanding the language model's head. The loss function is defined as follows:

$$\mathcal{L} = -\sum_{t=1}^{T}\sum_{i}^{V} y_{t,i}\log(\hat{y}_{t,i}), \tag{3}$$

where T represents the sequence length, and V denotes the vocabulary size.

Given the introduction of special tokens ranging from <nexa_0> to <nexa_N-1>, along with the distinct token <nexa_end>, which are absent in the Gemma-2B pretrained dataset, we confront an imbalanced dataset challenge during model training. To address this, we adopt a weighted cross-entropy loss as a surrogate loss to improve convergence:

$$\mathcal{L} = -\sum_{t=1}^{T}\sum_{i}^{V} \omega_i y_{t,i}\log(\hat{y}_{t,i}), \tag{4}$$

In our configuration, non-special tokens are assigned a weight of 1, while special tokens receive elevated weights. Early-stage training experiments indicate that increasing token weight can expedite convergence. The validation loss, based on Equation (3) with varying surrogate losses for training, is illustrated in Figure (6). Our findings suggest that employing a surrogate training loss early in the training process aids convergence. Nonetheless, experiments reveal no performance disparity in the fine-tuned model nor significant differences in wall-clock time. Therefore, utilizing an equal-weighted token loss is recommended for a small number of function tokens. In our benchmark tests, the evaluated model is trained by equal token weights.

5 Discussion and future works

Our current training initiative proves that any specific function can be encapsulated into a newly coined term, *functional token*, a novel token type seamlessly integrated into both the tokenizer and the model. This model, through a cost-effective training process amounting to merely two cents, facilitates the deployment of AI agents characterized by their remarkably low latency and high accuracy.

The potential impacts of our research are extensive. For application developers, including those at DoorDash and Yelp, our model paves the way for training on application-specific scenarios. Developers can pinpoint the APIs most utilized by their audience, transform these into functional tokens for the Octopus model, and proceed with deployment. This strategy has the capacity to fully

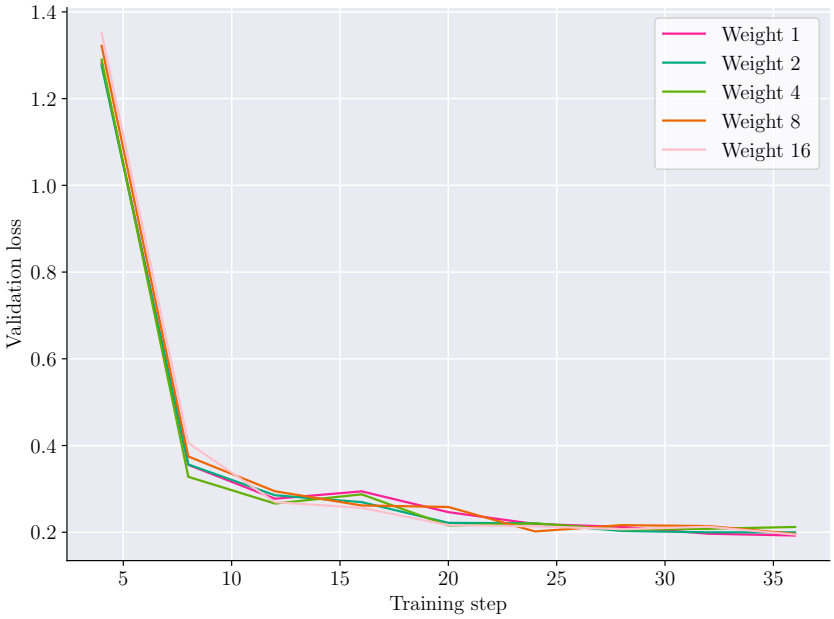

Figure 6: The validation loss using different surrogate loss functions with different weights for the special tokens.

automate app workflows, emulating functionalities akin to Apple's Siri, albeit with significantly enhanced response speeds and accuracy.

Furthermore, the model's application within operating systems of PCs, smartphones, and wearable technology presents another exciting avenue. Software developers could train minor LoRAs specific to the operating system. By accumulating multiple LoRAs, the model facilitates efficient function calling across diverse system components. For instance, incorporating this model into the Android ecosystem would enable Yelp and DoorDash developers to train distinct LoRAs, thus rendering the model operational on mobile platforms as well.

Looking ahead, we aim to develop a model dedicated to on-device reasoning. Our ambitions are dual-pronged: firstly, to achieve notable speed enhancements for cloud deployments, vastly outpacing GPT-4 in speed metrics. Secondly, to support local deployment, offering a valuable solution for users mindful of privacy or operational costs. This dual deployment strategy not only extends the model's utility across cloud and local environments, but also caters to user preferences for either speed and efficiency or privacy and cost savings.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[3] Erik Brynjolfsson, Danielle Li, and Lindsey R Raymond. Generative ai at work. Technical report, National Bureau of Economic Research, 2023.

[4] Rich Caruana. Multitask learning. *Machine learning*, 28:41–75, 1997.

[5] Imran Chaudhri. Humane ai, 2024. URL `https://humane.com/`. Accessed on March 31, 2024.

[6] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *Advances in Neural Information Processing Systems*, 36, 2024.

[7] Xin Luna Dong, Seungwhan Moon, Yifan Ethan Xu, Kshitiz Malik, and Zhou Yu. Towards next-generation intelligent assistants leveraging llm techniques. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 5792–5793, 2023.

[8] Yu Du, Fangyun Wei, and Hongyang Zhang. Anytool: Self-reflective, hierarchical agents for large-scale api calls. *arXiv preprint arXiv:2402.04253*, 2024.

[9] Div Garg. Multion ai, 2024. URL `https://www.multion.ai/`. Accessed on March 31, 2024.

[10] Gemma Team, Google DeepMind. Gemma: Open models based on gemini research and technology, 2023. URL `https://goo.gle/GemmaReport`.

[11] Allyson I Hauptman, Beau G Schelble, Nathan J McNeese, and Kapil Chalil Madathil. Adapt and overcome: Perceptions of adaptive autonomous agents for human-ai teaming. *Computers in Human Behavior*, 138: 107451, 2023.

[12] Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, et al. Metagpt: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.

[13] Yining Hong, Haoyu Zhen, Peihao Chen, Shuhong Zheng, Yilun Du, Zhenfang Chen, and Chuang Gan. 3d-llm: Injecting the 3d world into large language models. *Advances in Neural Information Processing Systems*, 36:20482–20494, 2023.

[14] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[15] Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. *arXiv preprint arXiv:2305.06983*, 2023.

[16] Zhanming Jie, Trung Quoc Luong, Xinbo Zhang, Xiaoran Jin, and Hang Li. Design of chain-of-thought in math problem solving, 2023.

[17] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

[18] Bo Li, Yuanhan Zhang, Liangyu Chen, Jinghao Wang, Fanyi Pu, Jingkang Yang, Chunyuan Li, and Ziwei Liu. Mimic-it: Multi-modal in-context instruction tuning. *arXiv preprint arXiv:2306.05425*, 2023.

[19] Huayang Li, Yixuan Su, Deng Cai, Yan Wang, and Lemao Liu. A survey on retrieval-augmented text generation. *arXiv preprint arXiv:2202.01110*, 2022.

[20] Yaobo Liang, Chenfei Wu, Ting Song, Wenshan Wu, Yan Xia, Yu Liu, Yang Ou, Shuai Lu, Lei Ji, Shaoguang Mao, et al. Taskmatrix. ai: Completing tasks by connecting foundation models with millions of apis. *arXiv preprint arXiv:2303.16434*, 2023.

[21] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023.

[22] Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, and Yang Liu. Prompt injection attack against llm-integrated applications. *arXiv preprint arXiv:2306.05499*, 2023.

[23] Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, et al. Mobilellm: Optimizing sub-billion parameter language models for on-device use cases. *arXiv preprint arXiv:2402.14905*, 2024.

[24] llama.cpp team. llama-cpp. Software available at `https://github.com/ggerganov/llama.cpp`, 2023. Accessed on March 31, 2024.

[25] David Luan. Adept ai, 2024. URL `https://www.adept.ai/`. Accessed on March 31, 2024.

[26] Jesse Lyu. Rabbit r1, 2024. URL `https://www.rabbit.tech/`. Accessed on March 31, 2024.

[27] Yuning Mao, Pengcheng He, Xiaodong Liu, Yelong Shen, Jianfeng Gao, Jiawei Han, and Weizhu Chen. Generation-augmented retrieval for open-domain question answering. *arXiv preprint arXiv:2009.08553*, 2020.

[28] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

[29] Bhargavi Paranjape, Scott Lundberg, Sameer Singh, Hannaneh Hajishirzi, Luke Zettlemoyer, and Marco Tulio Ribeiro. Art: Automatic multi-step reasoning and tool-use for large language models. *arXiv preprint arXiv:2303.09014*, 2023.

[30] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. *arXiv preprint arXiv:2305.15334*, 2023.

[31] Nikhil Pinnaparaju, Reshinth Adithyan, Duy Phung, Jonathan Tow, James Baicoianu, and Nathan Cooper. Stable code 3b, 2023. URL `https://huggingface.co/stabilityai/stable-code-3b`.

[32] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*, 2023.

[33] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. *OpenAI blog*, 2018.

[34] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

[35] Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhlgay, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. In-context retrieval-augmented language models. *Transactions of the Association for Computational Linguistics*, 11:1316–1331, 2023.

[36] Jingqing Ruan, Yihong Chen, Bin Zhang, Zhiwei Xu, Tianpeng Bao, Guoqing Du, Shiwei Shi, Hangyu Mao, Xingyu Zeng, and Rui Zhao. Tptu: Task planning and tool usage of large language model-based ai agents. *arXiv preprint arXiv:2308.03427*, 2023.

[37] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2024.

[38] Weizhou Shen, Chenliang Li, Hongzhan Chen, Ming Yan, Xiaojun Quan, Hehong Chen, Ji Zhang, and Fei Huang. Small llms are weak tool learners: A multi-llm agent. *arXiv preprint arXiv:2401.07324*, 2024.

[39] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems*, 36, 2024.

[40] Noah Shinn, Beck Labash, and Ashwin Gopinath. Reflexion: an autonomous agent with dynamic memory and self-reflection. *arXiv preprint arXiv:2303.11366*, 2023.

[41] Benjamin Spector and Chris Re. Accelerating llm inference with staged speculative decoding. *arXiv preprint arXiv:2308.04623*, 2023.

[42] Venkat Krishna Srinivasan, Zhen Dong, Banghua Zhu, Brian Yu, Damon Mosk-Aoyama, Kurt Keutzer, Jiantao Jiao, and Jian Zhang. Nexusraven: a commercially-permissive language model for function calling. In *NeurIPS 2023 Foundation Models for Decision Making Workshop*, 2023.

[43] Yashar Talebirad and Amirhossein Nadiri. Multi-agent collaboration: Harnessing the power of intelligent llm agents. *arXiv preprint arXiv:2306.03314*, 2023.

[44] Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, and Le Sun. Toolalpaca: Generalized tool learning for language models with 3000 simulated cases. *arXiv preprint arXiv:2306.05301*, 2023.

[45] Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

[46] MLC team. Mlc-llm. Software available at `https://github.com/mlc-ai/mlc-llm`, 2023. Accessed on March 31, 2024.

[47] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[48] Trelis. Llama-2-7b-chat-hf-function-calling-v3: A model fine-tuned for function calling. Hugging Face Model Repository, 2023. URL `https://huggingface.co/Trelis/Llama-2-7b-chat-hf-function-calling-v3`. Accessed on March 31, 2024.

[49] Junyang Wang, Haiyang Xu, Jiabo Ye, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. Mobile-agent: Autonomous multi-modal mobile device agent with visual perception. *arXiv preprint arXiv:2401.16158*, 2024.

[50] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *arXiv preprint arXiv:2308.11432*, 2023.

[51] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.

[52] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. Empowering llm to use smartphone for intelligent task automation. *arXiv preprint arXiv:2308.15272*, 2023.

[53] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155*, 2023.

[54] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.

[55] Daliang Xu, Wangsong Yin, Xin Jin, Ying Zhang, Shiyun Wei, Mengwei Xu, and Xuanzhe Liu. Llmcad: Fast and scalable on-device large language model inference. *arXiv preprint arXiv:2309.04255*, 2023.

[56] Zhao Yang, Jiaxuan Liu, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. Appagent: Multimodal agents as smartphone users. *arXiv preprint arXiv:2312.13771*, 2023.

[57] Yifan Yao, Jinhao Duan, Kaidi Xu, Yuanfang Cai, Zhibo Sun, and Yue Zhang. A survey on large language model (llm) security and privacy: The good, the bad, and the ugly. *High-Confidence Computing*, page 100211, 2024.

[58] Zhuosheng Zhang, Aston Zhang, Mu Li, and Alex Smola. Automatic chain of thought prompting in large language models. *arXiv preprint arXiv:2210.03493*, 2022.

Appendix

A.1 Android function examples

```
def take_a_photo(camera="back", resolution="1080p"):
    """
    Captures a photo using the specified camera and resolution settings.

    Parameters:
    - camera (str, optional): Specifies the camera to use. Can be 'front'
    or 'back'. The default is 'back'. Optional to provide.
    - resolution (str, optional): Sets the photo resolution. Options
    include '720p', '1080p', and '4K'. The default is '1080p'. Optional to
    provide.

    Returns:
    - str: The string contains the file path of the captured photo if
    successful, or an error message if not. Example: '/storage/emulated/0/
    Pictures/MyApp/IMG_20240310_123456.jpg'
    """


def get_trending_news(category=None, region='US', language='en',
    max_results=5):
    """
    Fetches trending news articles based on category, region, and language.

    Parameters:
    - category (str, optional): News category to filter by, by default use
    None for all categories. Optional to provide.
    - region (str, optional): ISO 3166-1 alpha-2 country code for region-
    specific news, by default, uses 'US'. Optional to provide.
    - language (str, optional): ISO 639-1 language code for article
    language, by default uses 'en'. Optional to provide.
    - max_results (int, optional): Maximum number of articles to return, by
     default, uses 5. Optional to provide.

    Returns:
    - list[str]: A list of strings, each representing an article. Each
    string contains the article's heading and URL.
    """


def get_weather_forecast(location, days=1):
    """
    Provides a weather forecast for a specified location over a given
    number of days. Each day's forecast includes a brief description of the
     expected weather conditions.

    Parameters:
    - location (str): The location for which the weather forecast is
    desired. Can be a city name, ZIP code, or other location identifiers.
    - days (int, optional): The number of days to include in the forecast,
    starting from today. The default is 1 day. Optional to provide.

    Returns:
    - list[str]: A list of strings, each representing the weather forecast
    for one day. Each string includes the date and a brief description of
    the weather conditions. Formatted in 'YYYY-MM-DD: Description' format.
```

```
    """


def send_email(recipient, subject, body, attachments=None, cc=None, bcc=
    None):
    """
    Sends an email with optional attachments, CC, and BCC.

    Parameters:
    - recipient (str): Primary recipient's email address.
    - subject (str): Email subject line.
    - body (str): Main email body content.
    - attachments (list of str, optional): A list of file paths
    representing files to attach to the email. Defaults to None, indicating
     no attachments. Optional to provide.
    - cc (list of str, optional): A list of email addresses to include in
    the Carbon Copy (CC) field. Defaults to None. Optional to provide.
    - bcc (list of str, optional): A list of email addresses to include in
    the Blind Carbon Copy (BCC) field. Defaults to None. Optional to
    provide.

    Returns:
    """


def search_youtube_videos(query, max_results=10, search_filter="Relevance"):

    """
    Searches YouTube for videos matching a query.

    Parameters:
    - query (str): Search query.
    - max_results (int, optional): Maximum number of search results, by
    default, use 10. Optional to provide.
    - search_filter (enum, optional): Filter for search results, chosen
    from 'Relevance', 'Upload date', 'View Count', 'Rating'. By default,
    use 'Relevance'. Optional to provide.

    Returns:
    - list[str]: A list of strings, each string includes video names and
    URLs.
    """
```

A.2 Vehicle function examples

```
def adjust_volume(volume_diff=None, set_value=None):
    """
    Adjusts the device's volume by a specified difference or sets it to a
    specified value. Only one operation can be performed at a time.

    Parameters:
    - volume_diff (int, optional): The amount to adjust the current volume
    by. Positive to increase, negative to decrease, optional to provide.
    - set_value (int, optional): The target volume level to set, in the
    range of 0 to 50, optional to provide.

    Note:
    - If both `volume_diff` and `set_value` are provided, only one will be
    considered based on the implementation's logic.
```

```
    Returns:
    - bool: True if the volume was adjusted successfully, False otherwise.
    """

def set_climate_temperature(zone, temperature):
    """
    Configures the temperature for a specific zone within the vehicle's
    climate control system.

    Parameters:
    - zone (str): The zone to set the temperature for ('driver', 'passenger
    ', 'rear').
    - temperature (int): The target temperature in Fahrenheit, within the
    range of 60 to 80 degrees.

    Returns:
    - bool: True if the temperature was set successfully, False otherwise.
    """

def adjust_seat_position(seat, position, distance):
    """
    Modifies the position of a specified seat by a certain distance.

    Parameters:
    - seat (str): The seat identifier ('driver', 'passenger').
    - position (str): The direction to adjust the seat in ('forward', '
    backward', 'up', 'down').
    - distance (int): The amount of adjustment in millimeters.

    Returns:
    - bool: True if the seat was adjusted successfully, False otherwise.
    """

def control_window(window, position, distance):
    """
    Adjusts a vehicle window's position by a specific distance.

    Parameters:
    - window (str): The window to control ('front left', 'front right', '
    rear left', 'rear right').
    - position (str): The direction to move the window ('up' or 'down').
    - distance (int): The distance to move the window, in millimeters.

    Returns:
    - bool: True if the window was adjusted successfully, False otherwise.
    """

def operate_sunroof(action, intensity=None):
    """
    Operates the sunroof with a specified action and optional intensity.

    Parameters:
    - action (str): The sunroof operation to perform ('open', 'close', '
    tilt').
    - intensity (int, optional): The degree to which the sunroof should be
    opened or tilted, as a percentage, optional to provide.

    Returns:
```

```
- bool: True if the sunroof was operated successfully, False otherwise.
"""
```